



Flash Sale Platform

SwiftDrop

Full-Stack Engineering Challenge

1. Background & Platform Context

SwiftDrop is an online flash sale marketplace that has quietly built one of the most loyal buyer communities in the region. The platform's proposition is simple and powerful: sourced directly from overseas suppliers, SwiftDrop releases limited quantities of high-quality imported goods at prices that are significantly lower than what any retail shelf can offer. Products range from electronics and home appliances to fashion accessories and lifestyle items, each batch genuinely scarce, each price genuinely compelling.

The way SwiftDrop works is through Events. An event is a scheduled sale session created by the platform's administrators. Each event has a name, a scheduled go-live time, and one or more items assigned to it. Each item carrying its own name, unit price, cover photo, and a fixed quantity that will never be restocked once it sells out. When an event is created, it enters a Locked state: the items and prices are visible to browsing customers, but purchasing is not yet possible. A live countdown clock on every event page ticks toward the scheduled opening time, building anticipation and driving customers to refresh and wait.

The moment an event goes live, the buying window opens. Customers who have been waiting sometimes hundreds of them, sometimes more attempt to purchase simultaneously. The first to complete a purchase secures their item. Once an item's stock reaches zero, it is gone. No backorders, no waitlists, no second chances in that event.

This model has made SwiftDrop extremely popular. And that popularity has become its most pressing engineering problem.

The Problem: SwiftDrop's current platform was built to handle a modest user base. As the community grew and word spread, a heavy load of users waiting for each event go-live has grown dramatically. Today, when a high-demand event opens, the platform receives an enormous surge of simultaneous requests within the first few seconds. The existing system was not designed for this. Customers experience slow responses, failed purchases, confusing errors, and even server crashes. Most critically, the underlying data has shown inconsistencies cases where the system sold more units than were actually available, and cases where genuine buyers received errors despite stock being present.

SwiftDrop has commissioned a complete rebuild of the platform. The new system must be architected from the ground up to handle the reality of what SwiftDrop has become: a high-

demand, event-driven marketplace where now 500 to 1,000 users compete for limited stock in real time, under the pressure of a countdown clock, at the exact same moment.

Your task is to design and build that complete platform.

2. Platform Roles

Role	Description
Admin	A trusted operator of the SwiftDrop platform. Admins manage the full event lifecycle: creating events, assigning items and quantities, scheduling go-live times, and monitoring sales in real time. Admins also manage the user base. Admin accounts are not created through public registration.
Customer	A registered buyer on the platform. Customers browse upcoming and live events, view item details and remaining stock, and attempt to purchase items when an event is open. Customers can view their own purchase history and manage their account profile.

Admin accounts must be seeded directly into the database or created by an existing Admin.

There must be no publicly accessible route that allows a user to register or promote themselves as an Admin.

3. Functional Requirements

The following requirements define the complete scope of what the platform must do. All requirements marked High are mandatory for a passing submission. Medium requirements contribute to scoring.

3.1 Authentication & Access

ID	Module	Requirement	Priority
FR-A01	Auth	A new user can register with a unique email address, a display name, and a password. The system must reject duplicate email addresses.	High
FR-A02	Auth	A registered user can log in with their email and password. A secure session token must be issued upon successful login.	High
FR-A03	Auth	A logged-in user can log out. Their session token must be invalidated immediately.	High

FR-A04	Auth	All purchase, profile, and order history actions must require an authenticated session. Unauthenticated requests must be rejected with a clear error.	High
FR-A05	Auth	All Admin actions (event management, user management) must require an Admin-role session. Non-admin requests to admin endpoints must be rejected.	High
FR-A06	Auth	A logged-in user can change their own password by providing their current password and a new password.	Medium

3.2 Event Management (Admin)

ID	Module	Requirement	Priority
FR-E01	Events	An Admin can create a new sale event by providing: an event name, event cover photo, a scheduled go-live date and time, and one or more items (each with a name, unit price, and stock quantity between 100 and 500 units).	High
FR-E02	Events	Once created, an event enters a Locked state and remains locked until its scheduled go-live time is reached.	High
FR-E03	Events	An Admin can edit the details of a Locked event (name, go-live time, item details, quantities). Editing a Live or Closed event is not permitted.	High
FR-E04	Events	An Admin can manually force-open a Locked event ahead of its scheduled time, or force-close a Live event.	Medium
FR-E05	Events	An Admin can view a dashboard showing all events, their current state (Locked / Live / Closed / Sold Out), total units sold per item, and total revenue per event.	High
FR-E06	Events	An Admin can deactivate a Customer account. Deactivated accounts must not be able to log in.	Medium

3.3 Marketplace (Customer)

ID	Module	Requirement	Priority
FR-M01	Marketplace	An authenticated customer can browse all events. Each event entry must show: event name, event cover photo, go-live time, status, and a list of its items with names and prices.	High
FR-M02	Marketplace	For Locked events, the event page must display a live countdown timer to the scheduled go-live time. The purchase action must not be available while the event is Locked.	High

FR-M03	Marketplace	For Live events, the event page must display the remaining stock quantity for each item. This count must reflect real-time changes without requiring a full page reload.	High
FR-M04	Marketplace	When an item's stock reaches zero, it must immediately display a Sold-Out status. The purchase action for that item must be disabled. No further purchase attempts for that item must be processed.	High
FR-M05	Marketplace	When all items in an event reach zero stock, the event must automatically transition to a Closed / Sold Out state.	High

3.4 Purchase Flow

ID	Module	Requirement	Priority
FR-P01	Purchase	An authenticated customer can initiate a purchase for a single item from a Live event. The system must atomically verify stock availability and reserve the unit before any further processing occurs.	High
ID	Module	Requirement	Priority
FR-P02	Purchase	If stock is not available at the moment of the purchase request, the system must reject the request immediately with a clear, user-friendly message. No partial processing should occur.	High
FR-P03	Purchase	Following a successful stock reservation, the customer must be presented with a payment confirmation step. On confirmation, the order is recorded as Confirmed. On cancellation, the reserved stock must be immediately released back.	High
FR-P04	Purchase	A customer must not be able to purchase the same item from the same event more than once. Duplicate attempts must be rejected with a clear message.	High
FR-P05	Purchase	The Purchase button must be disabled immediately upon click and must not re-enable until a response is received from the server, preventing duplicate submissions.	High
FR-P06	Purchase	All purchase outcome states must be clearly communicated to the customer: success, sold out, already purchased, or system error — using plain language, not technical error codes.	High

3.5 Order History & Profile

ID	Module	Requirement	Priority
----	--------	-------------	----------

FR-O01	Profile	A logged-in customer can view their complete purchase history: event name, item name, quantity, price paid, date, and order status.	High
FR-O02	Profile	A logged-in customer can view and update their display name.	Medium
FR-O03	Profile	A logged-in admin can view a paginated list of all customer accounts with their status (active / deactivated).	Medium

4. Non-Functional Requirements

These requirements define how well the system must perform, not what it must do. They are equally mandatory and will be tested during the judging session.

ID	Category	Requirement	Target
NFR-01	Concurrency	The purchase endpoint must produce zero data inconsistencies when hit by 1,000 simultaneous requests. Final sold count must never exceed the defined stock quantity for any item.	Zero oversell
NFR-02	Availability	The application server must remain responsive throughout a 30-second load test of 1,000 concurrent virtual users (setting up a Load Balancer).	Uptime ≥ 95%
NFR-03	Traffic Mgmt	The system must handle excess requests gracefully. Requests that exceed capacity must receive a	No 5xx under load
ID	Category	Requirement	Target
		structured error response, not a timeout or server crash.	
NFR-04	Response Time	The purchase endpoint must respond within 2 seconds under normal load (up to 100 concurrent users).	p95 ≤ 2,000ms
NFR-05	Security	Passwords must be stored as hashed values using an industry-standard algorithm. Session tokens must be signed and validated on every protected request.	No plaintext passwords
NFR-06	Security	All API endpoints must return structured JSON error responses. Raw stack traces, HTML error pages, and internal system details must never be returned to the client.	Structured JSON errors

NFR-07	Data Integrity	Payment confirmation and order creation must be handled atomically. A confirmed order must correspond to exactly one reserved stock unit. No order should exist without a corresponding stock decrement.	Atomic order creation
NFR-08	Code Quality	The codebase must follow a clear separation of concerns: routing layer, business logic layer, and data access layer must be distinct. No business logic inside route handlers.	Clean architecture
NFR-09	Documentation	The repository must include a README that covers: local setup steps, required environment variables, database seed instructions (events, items, Admin account), and how to start all services.	README complete

5. Required: High-Level Architecture Diagram

This is a mandatory deliverable. A solution that works but cannot explain why it works is not a complete engineering submission.

As part of your submission, you must produce a high-level architecture diagram that illustrates how your system is structured to handle the high-concurrency traffic problem described in Section 1. The diagram must be included in your repository (PNG or PDF) and be presented live to the judges.

Your diagram is not expected to be a detailed system design document. It must clearly show the path a purchase request takes from the moment a customer sends a request to the server — and specifically, where and how your architecture prevents the system from breaking under load.

Judges will ask you to walk through your diagram and explain your decisions. You must be able to explain why you chose your approach.

6. User Stories

The following stories describe what each role needs to be able to do on the platform. Use these as your feature checklist during development.

6.1 Authentication

ID	User Story
----	------------

US-A01	As a Customer, I want to register with my email, display name, and password.
US-A02	As a Customer, I want to log in with my email and password.
US-A03	As a Customer, I want to log out of my account.
US-A04	As a Customer, I want to change my password from my account settings.
US-A05	As an Admin, I want to be set up through the database, not through public registration.

6.2 Event Management

ID	User Story
US-E01	As an Admin, I want to create a new event with a name, cover photo, go-live time, and one or more items each with a name, price, and stock quantity.
US-E02	As an Admin, I want to see a countdown to the go-live time after creating an event.
US-E03	As an Admin, I want to edit the details of an event before it goes live.
US-E04	As an Admin, I want to manually open or close an event at any time.
US-E05	As an Admin, I want to view a dashboard that shows all events, their status, units sold, and revenue.
US-E06	As an Admin, I want to deactivate a customer account.

6.3 Marketplace

ID	User Story
US-M01	As a Customer, I want to browse all events and see their items, prices, and current status.
US-M02	As a Customer, I want to see a live countdown on an upcoming event so I know when it opens.
US-M03	As a Customer, I want to see the remaining stock update in real time while an event is live.
US-M04	As a Customer, I want to see a clear Sold Out label when an item's stock is exhausted.

6.4 Purchase

ID	User Story
US-P01	As a Customer, I want to click Buy Now and have my item reserved instantly before someone else takes it.
US-P02	As a Customer, I want to review and confirm my purchase on a payment screen before the order is finalised.
US-P03	As a Customer, I want to receive a clear confirmation or explanation for every purchase outcome.
US-P04	As a Customer, I want to be prevented from buying the same item in the same event more than once.
US-P05	As an Admin, I want to view the total units sold and revenue for each item in each event.

6.5 Profile & Order History

ID	User Story
US-PR01	As a Customer, I want to view my profile with my display name and email.
US-PR02	As a Customer, I want to update my display name.
US-PR03	As a Customer, I want to view all my past orders with their item names, event names, prices, dates, and statuses.

7. Submission Checklist

Verify your submission includes all of the following before the judging session:

✓	Deliverable
☐	A Git repository (GitHub or GitLab) accessible to judges, containing all source code.
☐	A README with: local setup instructions, all required environment variables, database seed command (creates events, items, and one Admin account), and instructions to start all services, Dockers etc.
☐	A high-level architecture diagram (PNG or PDF) in the repository and ready to be presented live.
☐	Functional and non-functional requirements implemented and demonstrable.
☐	A list of edge cases which the new platform solves.

- If your solution includes a Load Balancer, ensure your repository includes evidence of stress or load testing that validates it holds under pressure.

8. How You Will Be Judged

Your submission will be evaluated across five categories. Beyond the features you build, judges are paying close attention to the infrastructure decisions behind your solution. How your system is designed to receive, distribute, and control incoming traffic before it ever reaches your application logic. A platform that stays consistent and responsive when hundreds of requests arrive simultaneously, without overwhelming any single point of the system, is the hallmark of a production grade build under. That kind of thinking, and how well it is realized in your implementation, is the centerpiece of what judges are looking for.

Judging Category	Description	Weight
Functional Requirements (FR)	Correct implementation of all High and Medium priority functional requirements as defined in Section 3.	10%
Non-Functional Requirements (NFR)	System performance, security, concurrency handling, and reliability under load as defined in Section 4.	15%
Architectural Diagram	A clear, well-structured diagram showing how the system handles high-concurrency traffic. Must be included in the repository (PNG or PDF) and presented live to the judges.	20%
Design Principles	Application of separation of concerns and purposeful code structure, with design principles appropriate to the chosen stack (e.g. OOP, SOLID, modular architecture)	15%
Demo Presentation	Live demonstration of the complete running system. A proper Architectural Diagram combined with a working implementation of the concurrency solution (Proper traffic handling etc) accounts for 40% of total obtainable marks.	40%

Note: Judges will ask you to walk through your architecture diagram live and explain your design decisions. You must be able to articulate not just what you built, but why specifically how your Load Balancer and Rate Limiter implementation addresses the concurrency problem described in Section 1

9. A Note Before You Begin

The problem in this task is not the login page or the product listing. Those are straightforward to build. The real problem is the one that SwiftDrop's current platform could not solve, is what happens in the few seconds when an event opens and hundreds to thousands of people all try to buy the same limited items at the same time. Under a massive load of requests how to keep the platform resilient and functioning accurately.

Plan your data model and your purchase flow architecture before you write a single endpoint. Understand how incoming traffic will be distributed before it reaches your application code. Once those two decisions are made and understood by your whole team, the rest of the build will follow naturally.

Good luck. Read every section. Build the important parts first.
